Ride Sharing Platform (Uber/Lyft) - Interview Guide

This comprehensive guide covers the complete ride sharing platform design, from basic matching to advanced machine learning optimization. Perfect for system design interviews at companies building real-time marketplaces.

Key Interview Questions You Should Ask First

"How would you handle driver GPS updates at scale?" Expect an hour-long Kafka topic with 100K messages/second, Redis Geo for real-time driver locations, and PostgreSQL with PostGIS for persistent storage. Mention batching updates every 4 seconds to reduce load, and discuss how you'd handle stale driver locations (drivers who haven't updated in 30 seconds shouldn't be matched). Bonus points if you mention Google's S2 library for efficient geospatial indexing.

"How do you handle the matching problem in real-time?" Walk through the Redis Geo query to find drivers within 5 km, the ML ranking model (XGBoost) that considers ETA, driver rating, acceptance rate, and surge pricing, and the atomic Redis operation to prevent double-assignment. Show you understand the trade-offs between optimal matching (Hungarian algorithm) and greedy matching (faster, more scalable).

"What happens when a driver accepts but then cancels?" Discuss the cascading failure problem: the driver's acceptance rate drops, their ETA calculations become stale, and other riders keep getting matched to them. Show you understand circuit breakers, exponential backoff, and how to temporarily blacklist problematic drivers.

"How do you scale this to 1 million concurrent rides?" Discuss horizontal sharding by geohash, Kafka partitioning by region, read replicas for analytics, and why you'd deploy separate matching services per geographic region to reduce latency.

"What's the biggest bottleneck in this system?" Usually it's the matching algorithm's computational complexity or the database write throughput for GPS updates. Show you can identify the bottleneck, propose solutions, and understand the trade-offs between consistency and performance.

Quick Reference

Key Numbers:
- Driver GPS updates: 100K/second
- Matching latency: <500ms
- Driver search radius: 5 km
- Driver ETA: 2 minutes
- Surge multiplier: 1.5x-2.0x
- Cancellation rate: 5-10%

Critical Components:
- Redis Geo for real-time driver locations
- Kafka for GPS streams
- PostgreSQL + PostGIS for persistent storage
- ML ranking model (XGBoost)
- Circuit breakers for failure protection

Common Mistakes:
- Forgetting about driver cancellations
- Not handling stale GPS data
- Single-region architecture
- Ignoring surge pricing
- Not discussing monitoring

Related Topics

Similar systems: Food delivery (DoorDash), Package delivery, Taxi dispatch, Carpooling

Key technologies: Geospatial indexing (S2, H3), Stream processing (Kafka, Flink), Machine learning (XGBoost, TensorFlow), Circuit breakers, Distributed locks

---

Production Rich Systems - Interview Guide Index

Welcome to the Production Rich Systems interview guide. This collection covers real-world system design scenarios you'll encounter when building or interviewing for companies that operate at scale.

Available Guides

Ride Sharing Platform (Uber/Lyft)

Complete guide to building a real-time ride sharing platform. Covers driver matching, GPS tracking at scale, ETA calculation, surge pricing, and handling cancellations.

Key topics: Geospatial indexing, Real-time matching, Stream processing, Machine learning, Circuit breakers

Difficulty: Advanced

Time: 45-60 minutes

How to Use These Guides

1. **Start with the overview** - Each guide begins with a high-level overview and key numbers to know
2. **Ask the questions** - We provide questions you should ask before diving into solutions
3. **Understand the trade-offs** - Every decision has trade-offs. Show you can think through them
4. **Practice explaining** - Use the diagrams and code examples to practice explaining your design
5. **Adapt to your domain** - Principles apply to many systems beyond the specific example

Coming Soon

We're adding guides for:

- Food Delivery (DoorDash, Uber Eats)
- Package Delivery (Amazon, FedEx)
- Taxi Dispatch
- Carpooling
- Fleet Management

Contribution

Found a mistake? Have a suggestion? Want to add a guide? Contribute at production-rich.com

---

Ride Sharing Platform (Uber/Lyft) - Interview Guide

Key Interview Questions You Should Ask First

"How would you handle driver GPS updates at scale?" Expect an hour-long Kafka topic with 100K messages/second, Redis Geo for real-time driver locations, and PostgreSQL with PostGIS for persistent storage. Mention batching updates every 4 seconds to reduce load, and discuss how you'd handle stale driver locations (drivers who haven't updated in 30 seconds shouldn't be matched). Bonus points if you mention Google's S2 library for efficient geospatial indexing.

"How do you handle the matching problem in real-time?" Walk through the Redis Geo query to find drivers within 5 km, the ML ranking model (XGBoost) that considers ETA, driver rating, acceptance rate, and surge pricing, and the atomic Redis operation to prevent double-assignment. Show you understand the trade-offs between optimal matching (Hungarian algorithm) and greedy matching (faster, more scalable).

"What happens when a driver accepts but then cancels?" Discuss the cascading failure problem: the driver's acceptance rate drops, their ETA calculations become stale, and other riders keep getting matched to them. Show you understand circuit breakers, exponential backoff, and how to temporarily blacklist problematic drivers.

"How do you scale this to 1 million concurrent rides?" Discuss horizontal sharding by geohash, Kafka partitioning by region, read replicas for analytics, and why you'd deploy separate matching services per geographic region to reduce latency.

"What's the biggest bottleneck in this system?" Usually it's the matching algorithm's computational complexity or the database write throughput for GPS updates. Show you can identify the bottleneck, propose solutions, and understand the trade-offs between consistency and performance.

Quick Reference

Key Numbers:
- Driver GPS updates: 100K/second
- Matching latency: <500ms
- Driver search radius: 5 km
- Driver ETA: 2 minutes
- Surge multiplier: 1.5x-2.0x
- Cancellation rate: 5-10%

Critical Components:
- Redis Geo for real-time driver locations
- Kafka for GPS streams
- PostgreSQL + PostGIS for persistent storage
- ML ranking model (XGBoost)
- Circuit breakers for failure protection

Common Mistakes:
- Forgetting about driver cancellations
- Not handling stale GPS data
- Single-region architecture
- Ignoring surge pricing
- Not discussing monitoring

Related Topics

Similar systems: Food delivery (DoorDash), Package delivery, Taxi dispatch, Carpooling

Key technologies: Geospatial indexing (S2, H3), Stream processing (Kafka, Flink), Machine learning (XGBoost, TensorFlow), Circuit breakers, Distributed locks

---

Ride Sharing Platform (Uber/Lyft) - Interview Guide

Key Interview Questions You Should Ask First

"How would you handle driver GPS updates at scale?" Expect an hour-long Kafka topic with 100K messages/second, Redis Geo for real-time driver locations, and PostgreSQL with PostGIS for persistent storage. Mention batching updates every 4 seconds to reduce load, and discuss how you'd handle stale driver locations (drivers who haven't updated in 30 seconds shouldn't be matched). Bonus points if you mention Google's S2 library for efficient geospatial indexing.

"How do you handle the matching problem in real-time?" Walk through the Redis Geo query to find drivers within 5 km, the ML ranking model (XGBoost) that considers ETA, driver rating, acceptance rate, and surge pricing, and the atomic Redis operation to prevent double-assignment. Show you understand the trade-offs between optimal matching (Hungarian algorithm) and greedy matching (faster, more scalable).

"What happens when a driver accepts but then cancels?" Discuss the cascading failure problem: the driver's acceptance rate drops, their ETA calculations become stale, and other riders keep getting matched to them. Show you understand circuit breakers, exponential backoff, and how to temporarily blacklist problematic drivers.

"How do you scale this to 1 million concurrent rides?" Discuss horizontal sharding by geohash, Kafka partitioning by region, read replicas for analytics, and why you'd deploy separate matching services per geographic region to reduce latency.

"What's the biggest bottleneck in this system?" Usually it's the matching algorithm's computational complexity or the database write throughput for GPS updates. Show you can identify the bottleneck, propose solutions, and understand the trade-offs between consistency and performance.

Quick Reference

Key Numbers:
- Driver GPS updates: 100K/second
- Matching latency: <500ms
- Driver search radius: 5 km
- Driver ETA: 2 minutes
- Surge multiplier: 1.5x-2.0x
- Cancellation rate: 5-10%

Critical Components:
- Redis Geo for real-time driver locations
- Kafka for GPS streams
- PostgreSQL + PostGIS for persistent storage
- ML ranking model (XGBoost)
- Circuit breakers for failure protection

Common Mistakes:
- Forgetting about driver cancellations
- Not handling stale GPS data
- Single-region architecture
- Ignoring surge pricing
- Not discussing monitoring

Related Topics

Similar systems: Food delivery (DoorDash), Package delivery, Taxi dispatch, Carpooling

Key technologies: Geospatial indexing (S2, H3), Stream processing (Kafka, Flink), Machine learning (XGBoost, TensorFlow), Circuit breakers, Distributed locks

---

Ride Sharing Platform (Uber/Lyft) - Interview Guide

Key Interview Questions You Should Ask First

"How would you handle driver GPS updates at scale?" Expect an hour-long Kafka topic with 100K messages/second, Redis Geo for real-time driver locations, and PostgreSQL with PostGIS for persistent storage. Mention batching updates every 4 seconds to reduce load, and discuss how you'd handle stale driver locations (drivers who haven't updated in 30 seconds shouldn't be matched). Bonus points if you mention Google's S2 library for efficient geospatial indexing.

"How do you handle the matching problem in real-time?" Walk through the Redis Geo query to find drivers within 5 km, the ML ranking model (XGBoost) that considers ETA, driver rating, acceptance rate, and surge pricing, and the atomic Redis operation to prevent double-assignment. Show you understand the trade-offs between optimal matching (Hungarian algorithm) and greedy matching (faster, more scalable).

"What happens when a driver accepts but then cancels?" Discuss the cascading failure problem: the driver's acceptance rate drops, their ETA calculations become stale, and other riders keep getting matched to them. Show you understand circuit breakers, exponential backoff, and how to temporarily blacklist problematic drivers.

"How do you scale this to 1 million concurrent rides?" Discuss horizontal sharding by geohash, Kafka partitioning by region, read replicas for analytics, and why you'd deploy separate matching services per geographic region to reduce latency.

"What's the biggest bottleneck in this system?" Usually it's the matching algorithm's computational complexity or the database write throughput for GPS updates. Show you can identify the bottleneck, propose solutions, and understand the trade-offs between consistency and performance.

Quick Reference

Key Numbers:
- Driver GPS updates: 100K/second
- Matching latency: <500ms
- Driver search radius: 5 km
- Driver ETA: 2 minutes
- Surge multiplier: 1.5x-2.0x
- Cancellation rate: 5-10%

Critical Components:
- Redis Geo for real-time driver locations
- Kafka for GPS streams
- PostgreSQL + PostGIS for persistent storage
- ML ranking model (XGBoost)
- Circuit breakers for failure protection

Common Mistakes:
- Forgetting about driver cancellations
- Not handling stale GPS data
- Single-region architecture
- Ignoring surge pricing
- Not discussing monitoring

Related Topics

Similar systems: Food delivery (DoorDash), Package delivery, Taxi dispatch, Carpooling

Key technologies: Geospatial indexing (S2, H3), Stream processing (Kafka, Flink), Machine learning (XGBoost, TensorFlow), Circuit breakers, Distributed locks